Mysten Deepbook V3

Security Assessment

April 15th, 2025 — Prepared by OtterSec

Sangsoo Kang sangsoo@osec.io

Michał Bochnak embe221ed@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MDV-ADV-00 | Fee Accounting Inconsistency 6

OS-MDV-ADV-01 | Lack of Revoke Function 7

General Findings **8**

OS-MDV-SUG-00 | Inaccurate Swap Output Estimation 9

OS-MDV-SUG-01 | Rounding Error in Quote Calculation 10

OS-MDV-SUG-02 | Code Maturity 11

Appendices

Vulnerability Rating Scale **12**

Procedure **13**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Mysten Labs engaged OtterSec to assess the `deepbookV3` program. This assessment was conducted between March 12th and April 11th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified several vulnerabilities in the fee computation logic where, when paying fees in DEEP, a zero deep quantity causes the fee to be incorrectly calculated as base or quote (OS-MDV-ADV-00), and a lack of revoke functions for caps of balance manager (OS-MDV-ADV-01).

We also made recommendations for modifying the codebase to improve functionality and ensure adherence to coding best practices (OS-MDV-SUG-02), and highlighted the possibility of inaccurate results for large trades due to the `MAX_FILLS` cap, creating mismatches between estimated and actual execution (OS-MDV-SUG-00). Furthermore, setting a very small lot size may create rounding errors, reducing price granularity and resulting in inaccurate or unfair trade valuations (OS-MDV-SUG-01).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/MystenLabs/deepbookv3. This audit was performed against commit 2c04083.

A brief description of the program is as follows:

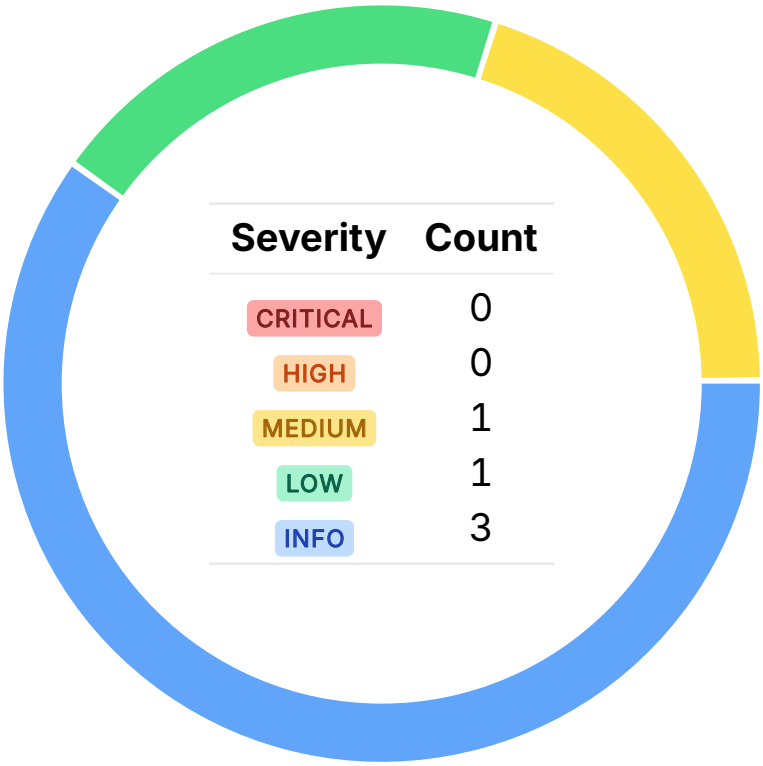
Name	Description
deepbookV3	A decentralized central limit order book (CLOB) built on the Sui blockchain, designed for high-performance, low-latency trading. It introduces features such as flashloans, governance, and improved account abstraction, leveraging Sui's parallel execution to enable efficient on-chain order matching.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	1
INFO	3

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MDV-ADV-00	MEDIUM	RESOLVED	If `fee_is_deep` is true but `deep_quantity` is 0, then the fee is calculated using either the base or the quote instead of DEEP.
OS-MDV-ADV-01	LOW	RESOLVED	`TradeCap` validates cap IDs against the allowed list, whereas `DepositCap` and `WithdrawCap` rely solely on `balance_manager_id`, rendering the revoke function ineffective.

© 2025 Otter Audits LLC. All Rights Reserved.

Fee Accounting Inconsistency MEDIUM OS-MDV-ADV-00

Description

When the user chooses to pay the fee in DEEP, `deep_quantity` is calculated from `fee_quantity`. However, if `deep_quantity` turns out to be 0, the fee may be incorrectly calculated in base or quote instead, resulting in the fee being paid in a way that does not match the user's intention.

>_ *deepbook/sources/vault/deep_price.move* RUST

```rust
public(package) fun fee_quantity([...]): Balances {
   [...]
    if (deep_quantity > 0) {
        balances::new(0, 0, deep_quantity)
    } else if (is_bid) {
        balances::new(
            0,
            math::mul(
                quote_quantity,
                constants::fee_penalty_multiplier(),
            ),
            0,
        )
    } [...]
}
```

Remediation

Ensure that if `fee_is_deep == true`, all fees must be converted to `DEEP`, even if the result is zero.

Patch

Fixed in PR#357.

© 2025 Otter Audits LLC. All Rights Reserved.

Lack of Revoke Function LOW OS-MDV-ADV-01

Description

`DepositCap` and `WithdrawCap` perform verification using only the `balance_manager_id` field, instead of checking the allowed list. However, `TradeCap` checks whether the cap ID is on the allowed list. As a result, it's not possible to remove the cap using the existing revoke function.

>_ *deepbook/sources/balance_manager.move* RUST

```rust
public(package) fun generate_proof_as_depositor(
    balance_manager: &BalanceManager,
    deposit_cap: &DepositCap,
    ctx: &TxContext,
): TradeProof {
    assert!(
        balance_manager.id() == deposit_cap.balance_manager_id,
        EDepositCapBalanceManagerMismatch,
    );

    TradeProof {
        balance_manager_id: object::id(balance_manager),
        trader: ctx.sender(),
    }
}
```

Remediation

Add the depositor and withdrawer to the allowed list in creation, and check that they're on the list during verification.

Patch

Fixed in PR#348.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MDV-SUG-00	`get_quantity_out` may return inaccurate results for large trades due to the absence of `MAX_FILLS` cap, creating mismatches between estimated and actual execution.
OS-MDV-SUG-01	Setting a very small `lot_size` may create rounding errors, reducing price granularity and resulting in inaccurate or unfair trade valuations.
OS-MDV-SUG-02	Recommendations to improve functionality and ensure adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Inaccurate Swap Output Estimation

OS-MDV-SUG-00

Description

`book::get_quantity_out` simulates trade execution without enforcing a cap, `MAX_FILLS`. However, in actual execution, `match_against_book` limits the number of orders through `MAX_FILLS`. This may result in discrepancies between simulation and actual execution, especially for large trades.

Remediation

Limit the number of matched orders in `get_amount_out` using `MAX_FILLS`.

Patch

Fixed in 0606ed9.

© 2025 Otter Audits LLC. All Rights Reserved.

Rounding Error in Quote Calculation OS-MDV-SUG-01

Description

`pool::create_permissionless_pool` allows the user to set any `tick_size`. The admin has the ability to adjust pool parameters or unregister pools to mitigate potential issues. However, if the `lot_size` is set too small, it can introduce rounding errors in quote calculations, especially with large order sizes. For instance, assuming SUI is $1, in the `SUI:USDC` pair where the lot size is 1 and the minimum size is 1000, both 1000 and 1999 units of base asset may map to the same quote value, creating pricing inaccuracies.

Remediation

Monitor the configuration of the permissionless pool.

Patch

Fixed in PR#349.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-MDV-SUG-02

Description

1. In whitelisted pools, `state::historic_maker_fee` is always zero, resulting in `fee_quantity` to be zero as well. As a result, both branches of the `if (!fill.expired())` block performs no meaningful actions related to fees. This renders the conditional check useless and may thus be removed.

   >_ *deepbook/sources/state/state.move* RUST

   ```rust
   fun process_fills(self: &mut State, fills: &mut vector<Fill>, ctx: &TxContext) {
       let mut i = 0;
       let num_fills = fills.length();
       while (i < num_fills) {
           [...]
           if (!fill.expired()) {
               fill.set_fill_maker_fee(&fee_quantity);
               self.history.add_volume(base_volume, account.active_stake());
               self.history.add_total_fees_collected(fee_quantity);
           } else {
               account.add_settled_balances(fee_quantity);
           };

           i = i + 1;
       };
   }
   ```

2. Replace the assert in `get_quantity_out` with one from `swap_exact_quantity` to improve clarity and semantics.

Remediation

Implement the above mentioned suggestions.

Patch

Issue #1 and #2 resolved in PR#342.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.